A+ Computer Science
Computer Science Competition
Hands-On Programming Set

I. General Notes

1. Do the problems in any order you like. They do not have to be done in order from 1 to 12.

2. All problems have a value of 60 points.

3. There is no extraneous input. All input is exactly as specified in the problem. Unless specified by the problem, integer inputs will not have leading zeros. Unless otherwise specified, your program should read to the end of file.

4. Your program should not print extraneous output. Follow the form exactly as given in the problem.

5. A penalty of 5 points will be assessed each time that an incorrect solution is submitted. This penalty will only be assessed if a solution is ultimately judged as correct.

II. Names of Problems

Number	Name
Problem 1	Albert
Problem 2	Bob
Problem 3	Cindy
Problem 4	Duke
Problem 5	Ella
Problem 6	Fernando
Problem 7	Gary
Problem 8	Hellen
Problem 9	Ian
Problem 10	Jacob
Problem 11	Kathleen
Problem 12	Louis

For more Computer Science practice tests and materials, go to www.apluscompsci.com

1. Albert

Program Name: Albert.java **Input File: albert.dat**

Albert is known for singing at the top of his lungs each morning! To give him some inspiration, his friends often whisper words and phrases into his ear. Being the singer he is, Albert will loudly recite whatever word or phrase is whispered into his ear. However, all this singing can become quite tiring after a while, so Albert has decided to produce a program that will simulate his lovely singing.

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases each consist of a single line of input denoting a word or phrase which has been whispered into Albert's ear by his friends. Note that each word or phrase will consist of upper and/or lowercase English letters separated by spaces.

Output

For each test case, print a single line containing the word or phrase that was whispered into Albert's ear by his friends followed by three exclamation points ("`!!!`").

Example Input File

```
3
Hey Albert
I love singing
Woohoo
```

Example Output to Screen

```
Hey Albert!!!
I love singing!!!
Woohoo!!!
```

2. Bob

Program Name: Bob.java **Input File: bob.dat**

Bob has been learning about computers in school, specifically about how calculators work. Bob's teacher told him that calculators do not understand normal infix expressions like humans do, such as 1 + 3 * 8, but rather that they use something called postfix to be able to evaluate expressions more efficiently. For example, a calculator would rather interpret the infix expression 1 + 3 * 8 as 3 8 * 1 +. Bob thought all of this was quite interesting, albeit quite complicated, so he decided that he wanted to design a postfix that can handle solving much simpler expressions. For example, if a postfix expression is in the form a b * or a b + etc., these would be equivalent to the infix expression a * b and a + b, respectively. Knowing this, Bob begins typing away at his computer to make his very own calculator!

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases will each consist of a single line containing two integers followed by an operator (either + or `*`), all separated by spaces. Each test case represents a single expression that Bob's calculator should be able to solve.

Output

For each test case, output a single line containing a single integer, the evaluation of the expression given in the test case.

Example Input File

```
3
2 3 +
10 8 *
100 0 *
```

Example Output to Screen

```
5
80
0
```

3. Cindy

Program Name: Cindy.java **Input File: cindy.dat**

Cindy's mother has tasked her with going to the store to buy bread. After giving her a wad of cash, her mother gave her the simple instructions of, "run along now and buy as many loaves as you can with this money." After reaching the grocery store and finally locating the bread aisle, Cindy is overwhelmed with all the price tags of sourdough, everything bagels, ciabatta loaves, and more! Being in quite the pickle, Cindy asks you to help her out. She has provided you with a list of prices of the distinct types of bread along with the amount of money she was entrusted with by her mother. Knowing these two things, along with the fact that Cindy can buy at most one of each type of loaf, can you help Cindy figure out the maximum number of loaves she can buy?

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases will each consist of two lines. The first of which will contain two integers, `n`, and `k`, denoting the number of types of bread in the bread aisle and the amount of money Cindy has, respectively. The second line will consist of `n` space-separated integers denoting the price of each type of bread.

Output

For each test case, output a single line containing a single integer, the maximum number of loaves of bread that Cindy can buy.

Example Input File

```
3
2 10
100 9
5 15
1 2 3 4 5
5 5
1 1 1 1 100
```

Example Output to Screen

```
1
5
4
```

4. Duke

Program Name: Duke.java **Input File: duke.dat**

Duke is a famous modern artist who specializes in making rectangular paintings consisting of a single color only. I know what you're thinking, "wow, how boring!", and while it may seem that way at first glance, it turns out that Duke's paintings regularly sell for millions at auction! The secret behind his paintings is that they are all computer generated, meaning that Duke only must specify the dimensions and color, and voila, a painting is instantly created! For some reason, though, Duke's painting program has been extremely sluggish, so he has asked you to do a complete redesign of his program for a cut of each painting's auction price, exciting!

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases will each consist of a single line containing two space-separated integers, `r`, and `c`, denoting the number of rows and columns, respectively, of the painting Duke wants to create. Following these two integers will be another space and then a character which denotes the desired fill color of Duke's painting.

Output

For each test case, output an `r` by `c` rectangle filled in with Duke's desired fill color, a perfect representation of Duke's modern art.

Example Input File

```
3
1 1 @
2 3 #
4 4 4
```

Example Output to Screen

```
@
###
###
4444
4444
4444
4444
```

5. Ella

Program Name: Ella.java **Input File: ella.dat**

Ella is a professional frog catcher, and with her trusty net she can gather up dozens of frogs every day. Both Ella and the frogs inhabit an infinite number line, where everything resides at integer tick marks along said number line. Ella begins each day at the point labeled 0 and walks rightwards until she hits her daily target location at some integer label that is greater than 0. Along the way, she catches every frog she sees using her net, that is, she catches any frog that resides between point 0 and her target location (inclusive). Of course, if she catches a frog on a previous day, she does not catch the same frog again on any succeeding day. Given information on the locations of all frogs (frogs reside at positive integer points and do not move over time) along with a list of daily target locations (in chronological order), can you help Ella calculate the number of frogs she will catch each day?

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases will each consist of three lines, the first of which containing two space-separated integers, `n`, and `m`, denoting the number of days Ella will go frog catching and the number of frogs on the number line, respectively. The second line will contain `n` space-separated integers denoting Ella's daily target locations. The third line will contain `m` space-separated integers denoting the positions of all frogs.

Output

For each test case, output `n` space-separated integers, denoting the number of frogs that Ella will catch each day she goes frog hunting.

Example Input File

```
3
3 3
1 2 3
1 2 3
5 3
10 20 30 40 50
5 10 15
2 2
100 1000
500 999
```

Example Output to Screen

```
1 1 1
2 1 0 0 0
0 2
```

6. Fernando

Program Name: Fernando.java **Input File: fernando.dat**

Fernando has a knack for mathematics but can be quite forgetful at times. To remedy this, he has been tasked with an exercise that combines both skills. Over a series of days, Fernando will be given a new integer each day. Upon receiving the latest integer, he needs to calculate the sum of the last three (or fewer, if three days have not yet passed) integers he has received and report it to his professor. Feeling a bit overwhelmed by this arduous task, he has asked you to write a program to verify his memory. You'll receive each day's new integer and will be responsible for reporting the relevant sums back to Fernando, or else his training will be utterly hopeless!

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases will each consist of two lines, the first of which will contain a single integer, `n`, denoting the number of days that Fernando's exercise will last. The second line of each test case will consist of `n` space-separated integers, representing the numbers that Fernando receives each day (in chronological order).

Output

For each test case, output `n` space-separated integers, denoting the sum of the previous three (or fewer, as aforementioned) integers that Fernando has received for each of the `n` days. This information should also be printed in chronological order.

Example Input File

```
3
10
1 1 1 2 2 2 3 3 3 4
8
1 2 1 2 1 2 1 2
2
100 200
```

Example Output to Screen

```
1 2 3 4 5 6 7 8 9 10
1 3 4 5 4 5 4 5
100 300
```

7. Gary

Program Name: Gary.java **Input File: gary.dat**

Gary recently discovered that he has superpowers! Was it that radioactive spider that bit him? Oh well, who cares, what matters now is that there's a cute cat trapped in a burning building! Gary only has a limited amount of time to rescue the adorable creature before the ultimate structural collapse of the building, which is laid out as a set of tiles in a rectangle consisting of `r` rows and `c` columns. Gary can move in and only in the four cardinal directions, but as mentioned, he has a superpower called dashing! Every `k` seconds since his last dash (and of course since the countdown initially begins), Gary can move forwards two tiles (instead of the usual one) in a single second. This doesn't mean, though, that Gary can phase through walls. If there is a wall in his path, he cannot dash through it and will instead come up just short of the wall like usual. If Gary chooses not to dash during a particular second, his dash will stay "charged" and can be saved for later. Knowing his abilities and the layout of the building, including the Gary's starting location and his goal location (where the cat is), you must report whether Gary will be able to reach the cat in time.

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases will each consist of a single line containing integers `r`, `c`, `k`, and `T`, representing the dimensions of the building, Gary's dash cooldown time, and the total time that Gary has, respectively, and then `r` lines, each containing `c` characters representing the building layout. Note that the `G` character denotes Gary's starting location, the `C` character represents the cat's location, `#` characters denote walls which Gary cannot pass through, and `.` characters represent free spaces which Gary can pass through.

Output

For each test case, output "`Dashing!`" (no quotes) if Gary can reach the cat in time (in less than or equal to `T` seconds) and "`Cat :(`" (no quotes) otherwise.

Example Input File

```
3
5 5 3 11
G#...
.#.#.
.#.#.
.#.#.
...#C
3 3 1 2
G..
..C
...
3 6 3 7
G#.C..
..##..
......
```

(Continued on next page)

Example Output to Screen

```
Cat :(
Dashing!
Dashing!
```

8. Hellen

Program Name: Hellen.java **Input File: hellen.dat**

Hellen oversees a large wind farm in Windville. The wind farm is laid out as a thin strip along a mountain, consisting of `n` different wind turbines, each of which help to provide power to Windville. However, wind conditions are quite unpredictable in the geographic area and there are other factors such as maintenance which may require Hellen to turn certain turbines on/off. Hellen has the power to toggle any individual wind turbine, meaning that she can turn it on if it is currently off, and off otherwise. Every day, she receives orders from the national wind advisors who provide her with an integer `k`, meaning that she should toggle all wind turbines numbered `k`, `2k`, `3k`, etc. (the numbering starts at 1). After a certain number of days, Hellen would like to create a report on the productivity of the Windville wind farm, and one statistic she would like to include is the number of wind turbines currently turned on (assume that all wind turbines are initially on). Can you help her keep track of all wind turbines?

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases will each consist of two lines, the first of which will contain two space separated integers, `n`, and `m`, denoting the number of wind turbines and the number of days, respectively. The second line will contain `m` space-separated integers, each of which represent the `k` values (in chronological order).

Output

For each test case, output a single line containing a single integer denoting the number of wind turbines that are left on after all `m` days have passed.

Example Input File

```
3
5 3
1 2 3
3 1
1
7 2
2 2
```

Example Output to Screen

```
3
0
7
```

9. Ian

Program Name: Ian.java **Input File: ian.dat**

Ian loves shopping, but he hates having to calculate how much he will have to pay at the checkout. There is always so much information to consider such as a coupon he would like to use, the amount of tax that he must pay, and of course the original price of what he would like to buy. But, like always, Ian yet again finds himself at the store buying a birthday present for his nephew. Since he is frugal, he would like to use a coupon to save some cash, and after that, the store will use the discounted price to apply tax of 10% to his subtotal. Can you help Ian figure out the final price of his nephew's present given its initial price, a coupon percentage, and the additional consideration of tax?

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases will each consist of a single line containing two space-separated floating-point numbers, `p`, and `c`, denoting the initial price of the item and the coupon value (`c` percent off), respectively.

Output

For each test case, output a single line containing a single floating-point number representing the final price of Ian's nephew's birthday present. Be sure to consider the coupon Ian applies and then the 10% tax. Finally, ensure that the value is rounded to two places after the decimal.

Example Input File

```
3
100.0 30.0
13.5 0.0
11.99 10.0
```

Example Output to Screen

```
77.00
14.85
11.87
```

10. Jacob

Program Name: Jacob.java **Input File: None**

Jacob loves smiley faces. Jacob loves smiling while drawing his favorite smiley faces. Jacob needs help creating a digital scan of his smiley face drawing (while smiling) so that he may smile while looking at his smiley face drawing on his computer. Please help him do so (it would make him smile)!

Input
None.

Output
A smiley face, exactly as shown below (no trailing whitespace).

Output to Screen
```
#############
#############
###○#####○###
#############
###○#####○###
####○○○○○####
#############
#############
```

11. Kathleen

Program Name: Kathleen.java **Input File: kathleen.dat**

Kathleen loves to travel, especially by airplane. She's currently in the process of planning her next summer vacation which will take her across the globe! To do so, she wants to perform a series of flights, however not all cities are directly connected by flights, of course. For example, it may be possible to get to Shang Hai from Austin by stopping in London, even if there is no direct path. Also, Kathleen lives in an ideal world where all flights that can happen in one direction can also happen in the other. For example, if there is a direct flight from San Francisco to Los Angeles, Kathleen may also fly directly from Los Angeles to San Francisco. As part of her trip planning, Kathleen would like to know if it is possible to, given a list of cities and potential flights, get from every city on her list to every other city through a series of flights. In other words, considering all pairs of cities on Kathleen's list, she would like to know if there is a path between each pair by following a series of direct flights. Can you help her out?

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases will each consist of multiple lines, the first of which consists of two space-separated integers, `n`, and `m`, which denote the number of cities on Kathleen's list and the number of potential direct flights, respectively. The next line will consist of `n` space-separated strings denoting the names of the cities on Kathleen's list. The next `m` lines will consist of 2 space-separated strings denoting a bidirectional direct flight between two cities on Kathleen's list. Note that no flights will be duplicated in the input, even with the order of cities swapped.

Output

For each test case, output a single line containing the string "`Yes`" (no quotes) if Kathleen's aforementioned reachability condition is satisfied, and "`No`" (no quotes) otherwise.

Example Input File

```
3
3 2
LosAngeles SanFrancisco Seattle
LosAngeles SanFrancisco
LosAngeles Seattle
1 0
Houston
5 4
ShangHai Boston NewYork Austin Denver
ShangHai Boston
Boston Austin
Austin Denver
Denver ShangHai
```

Example Output to Screen

```
Yes
Yes
No
```

12. Louis

Program Name: Louis.java **Input File: louis.dat**

Louis wants to a magician! To do so, he wants to impress his friends and family with all sorts of tricks and surprises. For example, he tells them to think of a number between 1 and `n` (inclusive) and then makes a series of guesses as to what the secret number is. Next, the friend or family member will tell Louis whether his guess was too high, too low, or exactly right. To their surprise, Louis guessed a hidden number between 1 and 1000 (inclusive) in only 10 guesses! This is a perfect illusion, because Louis can repeat this trick repeatedly and never use more than 10 guesses. The trick is that Louis knows the optimal solution for guessing the number in as few tries as possible. The question now is, do you? For a given value of n, Louis would like you to compute the highest possible number of guesses necessary (the worst case) it would take to find a hidden number (assuming that you guess in an optimal fashion, and that your friends and family are honest folks). Can you find the trick behind Louis's amazing display?

Input

The first line will contain a single integer `t` that indicates the number of test cases that follow. The next `t` test cases will each consist of a single line containing a single positive integer `n`, representing the upper bound (inclusive) on the number that is being guessed (where 1, of course, is the lower bound).

Output

For each test case, output a single line containing a single integer, the most guesses that it could possibly take to find the hidden number assuming that an optimal strategy is employed.

Example Input File

```
3
100
1000
4
```

Example Output to Screen

```
7
10
3
```